**Лекция 1: Концепции и парадигмы программирования**

*Вводная лекция к курсу «Объектно-ориентированное программирование на Python»*

*«Программы должны писаться для людей, которые будут их читать, а машины, которые будут эти программы исполнять — второстепенны.» — Harold Abelson*

Когда мы начинаем учиться программировать, мы часто думаем, что главное — выучить синтаксис (как писать цикл `for` или что такое `def`). Но по мере роста мастерства приходит понимание: программирование — это не про знание языка, а про **способ мышления**.

Сегодня мы поговорим о том, что такое парадигмы программирования. Это своего рода "линзы", через которые мы смотрим на задачу, или "стиль", в котором мы пишем музыку (код).

## 1. Понятия парадигма и концепция программирования

Начнем с определений.

**Концепция программирования** — это фундаментальная идея или подход, заложенные в основу языка или метода разработки. Например, концепция переменной (ячейки памяти с именем), концепция типа данных (целые числа, строки), концепция ветвления (если-иначе).

**Парадигма программирования** — это более глобальное понятие. Это совокупность концепций, определяющая **стиль написания программ** и **модель вычислений**. Это ответ на вопрос: "Как именно я буду думать о решении задачи?".

*Простая аналогия:* Представьте, что вы строите дом.

- **Концепция** — это кирпич, бетон, окно (строительный материал).
- **Парадигма** — это стиль архитектуры: барокко, хай-тек или классицизм. Материалы могут быть похожи, но подход к организации пространства и формы — кардинально разный.

## 2. Классификации парадигм

Мир парадигм многообразен, но их можно условно разделить на два огромных лагеря, из которых вырастают все остальные. Существует множество классификаций, но самая устойчивая делит парадигмы на **императивные** и **декларативные**.

*Если взглянуть на историю, то парадигмы развивались не линейно, а скорее как ветви дерева: от простых машинных инструкций к все более абстрактным моделям.*

| Категория | Суть | Примеры парадигм |
|---|---|---|
| **Императивные** | "Как это сделать?" — пошаговое описание изменения состояния программы. | Процедурная, ООП |
| **Декларативные** | "Что нужно получить?" — описание желаемого результата, а не процесса. | Функциональная, Логическая (SQL) |

## 3. Место парадигмы в разработке

Почему мы вообще должны об этом думать? Ведь код пишется строчка за строчкой.

Парадигма — это **инструмент абстракции**. Она позволяет справиться со сложностью реального мира.

- Если ваша задача — написать драйвер для микроконтроллера, где важна каждая миллисекунда, вам ближе императивный подход (чистая процедурщина).
- Если вы обрабатываете большой массив данных — вы интуитивно будете использовать декларативный стиль (как в SQL или при работе с pandas в Python).
- Если вы моделируете систему из взаимодействующих сущностей (студенты, преподаватели, курсы) — вам нужна объектно-ориентированная парадигма.

Выбор парадигмы (или их комбинации) — это первый шаг в проектировании архитектуры.

## 4. Императивная парадигма и ее виды

Слово "императив" означает "приказ". Императивная программа — это **последовательность приказов** (инструкций) компьютеру.

Представьте себе поваренную книгу:

1. Взять нож.
2. Нарезать лук.
3. Разогреть сковороду.
4. Налить масло.
5. Положить лук...

Программа работает точно так же: шаг за шагом, меняя состояние (лук был целым — стал резаным; сковорода была холодной — стала горячей).

**Виды императивной парадигмы:**

1. **Процедурное программирование:** Это прямой наследник первых языков (как Fortran или C). Мы группируем отдельные команды в блоки — **процедуры** или **функции**. Если нам нужно нарезать лук в трех местах рецепта, мы выносим это в отдельную функцию `нарезать_лук()`. Это помогает не повторяться (принцип DRY) и структурировать код.
2. **Объектно-ориентированное программирование (ООП):** Следующий шаг эволюции. Мы группируем не только команды, но и данные, к которым эти команды относятся, в один объект. Если процедурный стиль говорит: "У меня есть данные (лук) и функция (нарезать)", то ООП говорит: "У меня есть объект Лук, который умеет себя нарезать". Данные и поведение больше не разделены. (К этому мы вернемся подробно).

## 5. Декларативная парадигма и ее виды

Декларативное программирование — это программирование **"что", а не "как"**.

Продолжая кулинарную аналогию: декларативный подход — это не рецепт, а **меню в ресторане**. Вы заказываете "Омлет". Вы не объясняете повару, как разбивать яйца, как взбивать и как жарить. Вы просто декларируете желаемый результат. *Как* его достичь — это забота исполнителя (повара / компьютера).

**Виды декларативной парадигмы:**

1. **Функциональное программирование:** Программа строится как набор **чистых функций**. Ключевая идея: функции не должны менять состояние внешнего мира и зависеть от него. Если функция вызывается с одним и тем же аргументом, она

всегда возвращает один и тот же результат (как математическая функция `sin(x)`). Это исключает побочные эффекты, что делает программы надежными и легко тестируемыми. (Пример: Haskell, F#, частично современный Python).

2. **Логическое программирование:** Вы задаете систему фактов и правил, а затем задаете вопрос. Программа сама ищет путь логического вывода. Самая известная реализация — язык Prolog. *Пример:* Вы говорите: "Сократ — человек. Все люди смертны". Вы задаете вопрос: "Смертен ли Сократ?" Машина вывода говорит: "Да". Вы не писали алгоритм проверки смертности, вы описали факты.
3. **Программирование с помощью языков запросов (SQL):** Вы пишете `SELECT * FROM Users WHERE Age > 18`. Вы *декларируете*, что хотите получить всех совершеннолетних пользователей. Вы не описываете, как именно база данных будет сканировать таблицу, использовать индексы или сортировать данные. Это делает СУБД.

## 6. Выполнение императивных и декларативных программ

Как же компьютер понимает эти разные инструкции?

- **Императивная программа (пошаговая):** Здесь все относительно прямолинейно. Компилятор или интерпретатор превращает ваши команды в машинный код (или байт-код), и процессор выполняет их одну за другой. Состояние программы хранится в памяти и постоянно меняется. Вы, как программист, контролируете порядок.
- **Декларативная программа (описание результата):** Тут процесс интереснее. Компьютер не понимает "высокоуровневое" описание результата напрямую. В дело вступает **интерпретатор** или **движок**, который реализует свою внутреннюю *императивную* логику, чтобы достичь вашей декларативной цели.
    - Когда вы пишете SQL-запрос, движок базы данных строит **план запроса** (это и есть императивный алгоритм: сначала просканировать индекс по возрасту, потом подтянуть данные из таблицы).
    - Когда вы пишете код на функциональном языке, компилятор часто переписывает ваши абстракции в эффективные императивные инструкции для процессора.

**Итог:** Декларативное программирование — это уровень абстракции *над* императивным "железом".

## 7. Объектно-ориентированная парадигма

### *7.1. Что такое ООП?*

**Объектно-ориентированное программирование** — это парадигма, в которой программа рассматривается как совокупность взаимодействующих **объектов**, каждый из которых является экземпляром некоторого **класса** и обладает **состоянием** (данными) и **поведением** (методами).

Если процедурное программирование говорит: «У меня есть данные (структуры) и функции, которые этими данными манипулируют», то ООП объединяет данные и функции в единую сущность — объект. Объект сам знает, что он умеет делать, и сам отвечает за своё состояние.

*«Всё есть объект.» — Алан Кей*

*7.2. Краткая история: от моделирования к языкам*

Идеи, лежащие в основе ООП, начали формироваться в 1960-х годах.

- **1960-е: Simula**. Норвежские учёные Оле-Йохан Даль и Кристен Нигор создали язык Simula I (позже Simula 67), предназначенный для моделирования сложных систем. Им понадобился способ описывать объекты реального мира (корабли, очереди, процессы) так, чтобы они содержали и данные, и поведение. Так появились первые классы и объекты, наследование и динамическое связывание. Simula считается первым объектно-ориентированным языком, хотя сам термин тогда ещё не использовался.
- **1970-е: Smalltalk**. Язык, разработанный в Xerox PARC под руководством Алана Кея, сделал ООП своей единственной и главной идеей. В Smalltalk **всё** — объект, даже числа и классы. Именно Алан Кей ввёл термин «объектно-ориентированное программирование» и сформулировал ключевые принципы: объекты обмениваются сообщениями, каждый объект имеет свою память и свой набор операций.
- **1980-е: C++**. Бьёрн Страуструп создал расширение языка C, добавив в него классы и объекты (изначально язык назывался «C with Classes»). C++ стал мостом между эффективностью низкоуровневого C и удобством ООП, что обеспечило ему огромную популярность в промышленной разработке.
- **1990-е: Java и расцвет ООП**. Java был спроектирован как полностью объектно-ориентированный язык (хотя примитивные типы там всё же остались) с лозунгом «Write Once, Run Anywhere». Он популяризировал ООП среди широких масс разработчиков.

Сегодня ООП — одна из доминирующих парадигм в промышленной разработке, и большинство современных языков (C#, Java, C++, Python, PHP, JavaScript и др.) поддерживают её в той или иной форме.

*7.3. Основные понятия: объект, класс, атрибуты, методы*

Прежде чем говорить о принципах, давайте чётко определим базовые строительные блоки.

- **Класс** — это «чертёж», «шаблон» или «тип», по которому создаются объекты. Класс описывает, какие данные (атрибуты) будут у каждого объекта и какие действия (методы) объекты смогут выполнять.
    - *Аналогия:* Класс — это проект автомобиля на бумаге. Там указано, что у машины будут колёса, двигатель, руль, и что она умеет ехать и тормозить.
- **Объект** (или экземпляр) — это конкретный представитель класса, обладающий собственными значениями атрибутов.
    - *Аналогия:* Объект — это реальный автомобиль, собранный по этому проекту. У него есть конкретный цвет, конкретный номер двигателя, и он реально стоит на дороге. Можно завести много машин по одному чертежу — это будут разные объекты одного класса.
- **Атрибуты** (поля, свойства) — это данные, которые хранит объект. Они описывают его состояние.
    - *Пример:* Для объекта «автомобиль» атрибутами могут быть `скорость`, `марка`, `количество топлива`, `цвет`.

- **Методы** — это функции, принадлежащие объекту, которые определяют его поведение. Они могут изменять состояние объекта или взаимодействовать с другими объектами.
    - *Пример:* `автомобиль.завести()`, `автомобиль.ускориться(10)`, `автомобиль.затормозить()`.

На Python минимальный класс может выглядеть так (просто для иллюстрации, без глубокого погружения в синтаксис):

```
class Автомобиль:
    def __init__(self, марка, цвет):
        self.марка = марка          # атрибут
        self.цвет = цвет            # атрибут
        self.скорость = 0           # атрибут

    def поехать(self, ускорение):   # метод
        self.скорость += ускорение
        print(f"{self.марка} поехал со скоростью {self.скорость}")

# Создание объектов
машина1 = Автомобиль("Tesla", "красный")
машина2 = Автомобиль("Lada", "белый")
машина1.поехать(20)
```

### *7.4. Четыре столпа ООП*

Большинство авторов сходятся на том, что объектно-ориентированное программирование базируется на четырёх фундаментальных принципах: **абстракция**, **инкапсуляция**, **наследование** и **полиморфизм**. Иногда абстракцию рассматривают как часть инкапсуляции, но для ясности выделим её отдельно.

#### 7.4.1. Абстракция

**Абстракция** — это способ выделить важные характеристики объекта, отбросив второстепенные детали. Мы создаём упрощённую модель реальности, которая включает только то, что существенно для решения конкретной задачи.

- *Аналогия:* Когда вы садитесь в автомобиль, вы видите руль, педали, рычаг КПП. Вы *абстрагируетесь* от устройства двигателя, системы охлаждения, электроники — вам достаточно знать, как управлять машиной, чтобы ехать. Для водителя важна абстракция «автомобиль» с интерфейсом «руль-педали».
- *В программировании:* Если мы пишем систему учёта сотрудников, нам не нужно хранить всю информацию о человеке (цвет глаз, любимое блюдо). Мы выделяем абстракцию «Сотрудник» с атрибутами `имя`, `должность`, `оклад` и методами `повысить_зарплату()`, `перевести_в_отдел()`.

Абстракция позволяет справляться со сложностью, скрывая ненужные подробности и оставляя только то, что нужно для взаимодействия.

7.4.2. Инкапсуляция

**Инкапсуляция** — это объединение данных и методов, работающих с этими данными, внутри объекта, а также **сокрытие внутренней реализации** от внешнего мира. Объект предоставляет только строго определённый интерфейс (публичные методы) для взаимодействия с собой и защищает свои внутренние данные от некорректного изменения извне.

- *Аналогия:* Телефон. Вы нажимаете кнопку (метод), чтобы позвонить. Как внутри работает процессор, как передаётся сигнал — вас не касается. Вы не можете напрямую изменить напряжение в аккумуляторе — это внутреннее состояние. Производитель инкапсулировал сложность, оставив вам простой интерфейс.
- *В программировании:* В Python нет строгих модификаторов доступа (`private`, `protected`), как в Java или C++, но есть соглашения: атрибут с одним подчёркиванием (`_скорость`) считается защищённым (не для внешнего использования), а с двумя подчёркиваниями (`__скорость`) включает механизм *name mangling*, который затрудняет прямой доступ извне. Главная цель — предотвратить случайное изменение данных в неверном состоянии.

```
class БанковскийСчет:
    def __init__(self, начальный_баланс):
        self.__баланс = начальный_баланс   # инкапсулированное поле

    def пополнить(self, сумма):
        if сумма > 0:
            self.__баланс += сумма

    def получить_баланс(self):
        return self.__баланс
```

Инкапсуляция даёт:

- **Контроль доступа:** можно проверять корректность новых значений.
- **Слабое связывание:** изменение внутренней реализации не ломает код, который использует объект, если интерфейс остался тем же.

7.4.3. Наследование

**Наследование** — это механизм, позволяющий создать новый класс (потомок, подкласс) на основе существующего (родителя, суперкласса). Потомок автоматически получает все атрибуты и методы родителя (кроме приватных) и может добавить свои или переопределить унаследованные.

- *Аналогия:* Есть общий класс «Транспортное средство» (двигатель, колёса, скорость). От него можно породить класс «Автомобиль» (добавить багажник,

количество дверей) и класс «Мотоцикл» (добавить руль, отсутствие дверей). Они разделяют общую функциональность, но имеют свои особенности.

- *В программировании:* Это мощный инструмент для повторного использования кода и построения иерархий. Если у вас есть класс `Животное` с методом `дышать()`, то классы `Кошка` и `Собака` могут унаследовать этот метод, не дублируя код.

```
class ТранспортноеСредство:
    def __init__(self, скорость):
        self.скорость = скорость

    def двигаться(self):
        print("Транспорт движется")

class Автомобиль(ТранспортноеСредство):
    def __init__(self, скорость, марка):
        super().__init__(скорость)   # вызов конструктора родителя
        self.марка = марка

    def сигналить(self):
        print("Бип-бип!")

# Автомобиль умеет и двигаться (унаследовано), и сигналить (своё)
```

Важно: Наследование создаёт отношение «is-a» (является). Автомобиль *является* транспортным средством.

7.4.4. Полиморфизм

**Полиморфизм** (от греч. «много форм») — это способность объектов с одинаковым интерфейсом иметь различную реализацию. Один и тот же метод может вести себя по-разному в зависимости от того, к какому классу принадлежит объект.

- *Аналогия:* Команда «Нарисовать()». Для круга она означает одно, для квадрата — другое, для дома — третье. Но для того, кто отдаёт команду, важно лишь то, что все они умеют рисоваться.
- *В программировании:* Полиморфизм позволяет писать обобщённый код, который работает с объектами разных классов, если они поддерживают нужный метод. В языках со статической типизацией (Java, C++) для этого нужно явное наследование или интерфейсы. В Python, благодаря *утиной типизации*, полиморфизм достигается проще: «если объект крякает как утка и ходит как утка, значит это

утка» — важен только факт наличия метода, а не принадлежность к конкретному классу.

```
class Кошка:
    def голос(self):
        print("Мяу")

class Собака:
    def голос(self):
        print("Гав")

def позвать(животное):
    животное.голос()   # полиморфный вызов

позвать(Кошка())  # Мяу
позвать(Собака()) # Гав
```

Полиморфизм позволяет создавать гибкие расширяемые системы: вы можете добавлять новые классы, не меняя код, который их вызывает, если новые классы реализуют ожидаемый интерфейс.

*7.5. Сообщения: как объекты взаимодействуют*

В классическом определении ООП (Smalltalk) объекты взаимодействуют, посылая друг другу **сообщения**. Когда один объект хочет, чтобы другой что-то сделал, он отправляет ему сообщение. Получатель сам решает, как обработать это сообщение (какой метод выполнить). В современных языках это выглядит как вызов метода: `объект.метод(аргументы)`, но идеологически это именно посылка сообщения.

Это важный момент: объект не «вызывает функцию другого объекта», а просит его что-то сделать. Контроль остаётся у получателя.

*7.6. Преимущества и недостатки ООП*

Как и любая парадигма, ООП не является панацеей. Давайте объективно посмотрим на её сильные и слабые стороны.

| **Преимущества** | **Недостатки** |
|---|---|
| **Модульность:** Объекты — естественные модули, их легче разрабатывать, тестировать и отлаживать независимо. | **Сложность:** Проектирование иерархий классов требует опыта; неправильная архитектура может привести к запутанному коду. |

| Преимущества | Недостатки |
| --- | --- |
| **Повторное использование:** Наследование и композиция позволяют использовать готовый код в новых проектах. | **Накладные расходы:** Объекты потребляют больше памяти, а динамический полиморфизм (виртуальные функции) может быть медленнее прямых вызовов. |
| **Гибкость и расширяемость:** Полиморфизм позволяет легко добавлять новые типы, не меняя существующий код. | **Избыточность:** Для маленьких программ ООП часто избыточно — простой процедурный код был бы понятнее и быстрее. |
| **Соответствие реальному миру:** Моделирование сущностей предметной области упрощает понимание программы для заказчика и разработчика. | **Сложность параллелизма:** Объекты с изменяемым состоянием тяжело использовать в многопоточных средах без дополнительных синхронизаций. |

## Заключение

Ни одна парадигма не является панацеей. Python является **мультипарадигмальным** языком. Вы можете писать на нем простые процедурные скрипты, можете использовать функции как объекты (функциональный стиль), а можете строить огромные иерархии классов (ООП).

Наша задача на курсе — научиться мыслить категориями объектов и классов там, где это уместно. Мы будем учиться не просто писать код, а проектировать взаимодействие сущностей.